FIR Filter Project Write-Up

1 Design Approach

The design process started in MATLAB to make sure the filter met the required frequency specifications before moving into hardware. Instead of guessing parameters, the script incrementally increased the number of taps until the quantized filter achieved at least 80 dB stopband attenuation. This approach makes the hardware implementation already valid before any SystemVerilog code is written.

Once the coefficients were finalized, they were converted into fixed-point format so they could be used directly in hardware. This is important because hardware cannot efficiently use floating-point values, so everything must be scaled and represented using integers. The MATLAB script handled this conversion and exported the values into a file (matlab.hex) for use in the SystemVerilog modules.

Choosing the correct bit widths was one of the most important parts of the project. The inputs and outputs were set to 16 bits to balance precision and hardware cost, while the coefficients used 24 bits to meet the strict 80 dB attenuation requirement. Previously, the coefficients were set to 16 bits, but this did not meet the stopband attenuation requirement.

The product and accumulator widths were calculated to avoid overflow during multiply-accumulate operations. Since FIR filters sum many products together, the accumulator can grow large quickly. The MATLAB script estimated the worst-case values and made sure that the accumulator width was large enough to safely handle all operations without losing data. This directly impacts hardware reliability. If the bit widths are too small, the output becomes incorrect. If they are too large, unnecessary hardware resources are used.

For the filter design, the Parks-McClellan algorithm (firpm in MATLAB) was used. This decision was made because the project required a strict stopband attenuation of at least 80 dB, which is difficult to achieve efficiently with simpler design methods. The Parks-McClellan algorithm is used to design optimal FIR filters by minimizing the maximum error across frequency bands, rather than the average error. This results in a response where the ripple is evenly distributed in both the passband and stopband. This behavior is ideal when tight specifications are required.

In this project, the algorithm allowed precise control over:

Passband edge (0.2π)
Stopband edge (0.23π)
Weighting between passband and stopband errors

By assigning a higher weight to the stopband region, the design prioritizes achieving the required attenuation while maintaining acceptable passband behavior.

2 Architecture Trade-offs

Each FIR architecture was implemented to explore different hardware tradeoffs. The baseline design is the simplest and easiest to understand. It processes one sample per clock cycle using a straightforward multiply and accumulate structure. However, because all operations happen in a single stage, the critical path is long, which limits the maximum clock frequency.

The pipelined design improves this issue by inserting registers between operations. This reduces the amount of logic per stage and allows the design to run at a much higher clock frequency. The downside is that more registers are required, which increases resource usage.

Parallel designs differ from both approaches because they attempt to process multiple samples at the same time. While this increases throughput, it also requires significantly more hardware, especially multipliers. In practice, this led to high resource usage and limited improvement in clock speed.

3 Implementation Challenges

One of the main challenges in this project was dealing with FPGA resource limitations. The parallel FIR implementations required a large number of multipliers, which are mapped to DSP blocks on the FPGA. When compiling these designs, errors occurred due to insufficient DSP resources on the device: 5CGXFC7B6M15C6. This issue was resolved by selecting a different FPGA device with more available DSP blocks, such as the 5CGXFC9D6F27C7 device used in the final implementation. This highlights an important real-world constraint, which is that even if a design is logically correct, it may not be physically implementable on a given device due to hardware limitations.

4 Performance Analysis

The table below summarizes the hardware performance of each FIR architecture:

Metric	FIR_baseline	FIR_pipeline:	FIR_parallel_L2	FIR_parallel_L3	FIR_parallel_L3_pipeline
Logic Utilization (in ALMS)	13,814/113,560 (12%)	8,179/113,560 (7%)	26,434/113,560 (23%)	38,438/113,560 (34%)	38,312/113,560 (34%)
Total Registers	3249	9637	3249	3265	3344
Fmax	36.14 MHz	98.35 MHz	35.33 MHz	34.52 MHz	44.91 MHz
Total Thermal Power Dissipation	527.39 mW	527.34 mW	529.01 mW	529.90 mW	529.93 mW
Core Static Thermal Power Dissipation	519.61 mW	519.60 mW	519.63 mW	519.64 mW	519.64 mW
I/O Thermal Power Dissipation	7.78 mW	7.73 mW	9.38 mW	10.26 mW	10.28 mW

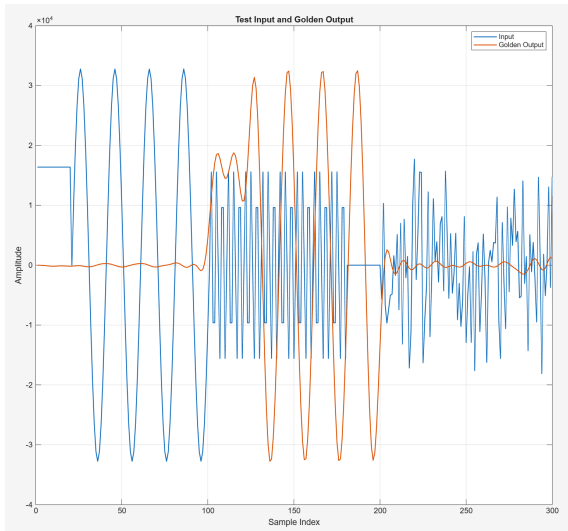
The results show clear differences between the architectures. The pipelined design achieved the highest clock frequency, reaching 98.35 MHz, which is significantly higher than the baseline(36.14 MHz). This confirms that reducing the critical path is effective for improving speed.

The parallel designs, while increasing hardware usage, did not significantly improve the clock frequency. Instead, they mainly increased the number of logic elements and DSP usage. This suggests that parallelization alone is not always the best approach unless higher throughput is specifically required.

Power consumption remained relatively constant across all designs. This is likely because the static power dominates, and the additional switching activity from different architectures does not significantly change the total power.

5 Verification

To check accuracy, the hardware output was compared against a reference output generated in MATLAB. The same quantized coefficients and input signal were used in both environments, allowing for a direct comparison. The testbench reads input samples from a file and checks the output against the expected golden values.



This approach makes sure that the hardware implementation matches the original filter design. By using file-based verification, it becomes easy to test different inputs and validate the design without manually checking waveforms.

```
# Loaded 300 input samples and 300 golden output samples.
# ======== FINAL RESULTS ========
#
# BASELINE FIR:
# Total outputs checked : 299
# Mismatch count        : 0
# Maximum absolute error: 0
# RESULT                : PASS
#
# PIPELINED FIR:
# Total outputs checked : 300
# Mismatch count        : 0
# Maximum absolute error: 0
# RESULT                : PASS
#
# PARALLEL FIR L=2:
# Total outputs checked : 300
# Mismatch count        : 0
# Maximum absolute error: 0
# RESULT                : PASS
#
# PARALLEL FIR L=3:
# Total outputs checked : 300
# Mismatch count        : 0
# Maximum absolute error: 0
# RESULT                : PASS
#
# PARALLEL FIR L=3 + PIPELINE:
# Total outputs checked : 300
# Mismatch count        : 0
# Maximum absolute error: 0
# RESULT                : PASS
# ==============================
```

6 Conclusion

Overall, this project demonstrates the full workflow of digital signal processing implementation, from algorithm design to hardware realization. It shows the importance of balancing accuracy, performance, and hardware constraints. The final designs show that pipelining is the most effective method for improving speed, while parallelization must be used carefully due to its high resource cost.